

हेलो! आपका स्वागत है **Unit 6: Functions** की complete class में.

मैंने आपके workspace में, unit 6 functions folder के अंदर ये study files create कर दी हैं:

- Study Guide File: chapter6_functions.md
- Code Examples File: functions_examples.js
- Practice Exercises File: functions_practice.js

चलिए अब functions के concepts को एकदम clean and simple Hinglish explanation के साथ पढ़ना शुरू करते हैं.

PART 1: BASIC FUNCTIONS AND DECLARATION

Functions JavaScript के core building blocks हैं. इनका मुख्य काम होता है code chunks को encapsulate करना ताकि हम उन्हें reusable बना सकें.

1. Function Kya Hai?

What is it? Function JavaScript statement का एक collection block है, जो एक specific task complete करता है.

Why is it needed? बिना functions के, अगर हमें एक ही काम 10 बार करना हो, तो हमें वही line बार-बार लिखनी पड़ेगी. Functions हमें DRY (Don't Repeat Yourself) principle follow करने में मदद करते हैं, जिससे code simple और structured रहता है.

How does it work? हम function को structure define करते हैं. जब तक हम उसे explicit command देकर execute (call) नहीं करते, तब तक उसका code नहीं चलता.

Syntax of Function Declaration

javascript

```
function functionName(parameters) {  
    // code statement block  
}
```

Syntax of Function Calling (Invocation)

javascript

```
functionName(arguments);
```

Simple example

javascript

```
function greetUser() {  
    console.log("Welcome Student!");  
}
```

```
greetUser(); // Calling the function
```

Output Welcome Student!

Step-by-step explanation Program execution parses the function definition. When it triggers greetUser(), compiler execute print functions inside body and returns back.

2. Parameters vs Arguments

What is it?

- **Parameters:** Variable list names coordinates structure setup placeholders functions define definitions list.
- **Arguments:** Actual concrete values pass variables values call function trigger execution.

Why is it used? Functions code inputs change parameters lists dynamic templates execution structure settings.

Example

javascript

```
function addNumbers(num1, num2) { // num1 and num2 are Parameters  
    console.log(num1 + num2);  
}
```

```
addNumbers(15, 25); // 15 and 25 are Arguments
```

Output 40

3. The return Keyword

What is it? return statement functions values computations evaluations variables dynamic updates outputs calculations caller pass.

Why is it needed? Agar function sirf logs output print parameters values, we cannot use it elsewhere in calculations. return allows us to fetch that calculation back.

How does it work? Function execution breaks immediately on hitting return value; instruction line and transfers outputs back.

Simple example

javascript

```
function getSquare(num) {  
    return num * num;  
}
```

```
let res = getSquare(5); // res gets the returned value 25
```

```
console.log("Result is: " + res);
```

Output Result is: 25

Common mistakes Return state line statement ke bad additional properties checks statements run commands likhna.

javascript

// Galti:

```
function test() {  
    return 10;  
    console.log("This will never print!"); // Return line completes scope immediately  
}
```

PART 2: FUNCTION EXPRESSIONS & ANONYMOUS FUNCTIONS

1. Function Expressions

What is it? Function blocks properties declarations code assign logic directly using variables templates variable structure assignments.

Why is it needed? Function levels scopes assignments, object properties handling dynamic variables settings.

Syntax

javascript

```
let myExpression = function() {  
    // code block
```

```
};
```

Simple example

javascript

```
let calcTriple = function(val) {  
    return val * 3;  
};
```

```
console.log(calcTriple(10));
```

Output 30

2. Anonymous Functions

What is it? A function without a name (bina naam ka function).

Why is it used? Immediate function passing arrays mappings, inline execution operations call checks.

Difference between Function Declaration and Function Expression

- **Function Declaration:** Hoisted behaviour sets (means declarations parses first, we can call them before writing them in file).
 - **Function Expression:** Non-hoisted parameters (you cannot call variables expressions before definitions line initialization).
-

PART 3: ARROW FUNCTIONS

1. Arrow Functions (() => {})

What is it? Arrow function (ES6 feature) ek clean syntactic shorthand representation code block design pattern hai.

Why is it used? Write clean code templates, remove function keyword, improve callbacks setups.

Normal Function vs Arrow Function

- **Normal Function:**

javascript

```
function double(x) {
```

```
    return x * 2;
}
```

- **Arrow Function:**

javascript

```
const double = x => x * 2;
```

Basic syntax

- If parameter count is 1: parenthesis () is optional.
- If body has only 1 line return expression: curly braces {} and return keyword are optional.

Examples

javascript

```
const greet = () => "Hello Everyone";
```

```
const sum = (a, b) => a + b;
```

```
console.log(greet());
```

```
console.log(sum(50, 50));
```

Output Hello Everyone 100

PART 4: DEFAULT, REST PARAMETERS & SPREAD BASICS

1. Default Parameters

What is it? Argument pass command parameters values undefined default setup check triggers standard initial values variables.

Why is it needed? Arguments pass status verification missing error limits skip.

Example

javascript

```
function greetUser(username = "Friend") {
```

```
    console.log("Hi " + username);
```

```
}
```

```
greetUser();    // Hi Friend
```

```
greetUser("Rahul"); // Hi Rahul
```

2. Rest Parameters (...args)

What is it? Pass multiple variables arguments array format dynamic inputs single container merge structures.

Why is it used? Calculations lists unknown sizes variables parameters inputs checks run.

Example

javascript

```
function listAnimals(...animals) {  
    console.log(animals); // real array representation  
}
```

```
listAnimals("Dog", "Cat", "Lion");
```

Output ["Dog", "Cat", "Lion"]

3. Spread Basics

What is it? Unpacking variables lists arrays elements values duplicate structures sets parameters coordinates maps.

Example

javascript

```
let letters = ["a", "b"];  
let allLetters = [...letters, "c", "d"];  
console.log(allLetters);
```

Output ["a", "b", "c", "d"]

PART 5: SCOPES IN JAVASCRIPT

Scope checks boundary limits visibility parameters configurations variables accesses.

1. Scopes Types

- **Global Scope:** Variables defined outside functions. Accessible everywhere in file.
- **Function Scope:** Variables declared inside functions. Accessible only inside that function.

- **Block Scope:** Variables declared with let or const inside curly braces {} (e.g., if, loops). Not accessible outside those brackets.

Example Scopes Check

javascript

```
let activeUser = "Ravi"; // Global
```

```
function processData() {
```

```
  let localScope = 100; // Function Scope
```

```
  if (true) {
```

```
    let insideBlock = "Secure"; // Block Scope
```

```
    console.log(activeUser); // OK
```

```
  }
```

```
  // console.log(insideBlock); // Error (Block scoped variables isolated)
```

```
}
```

PART 6: ADVANCED FUNCTION MODULES

1. Nested Functions

What is it? Defining functions inside functions containers.

Example

javascript

```
function father() {
```

```
  let cash = 500;
```

```
  function son() {
```

```
    console.log("Son spends father's cash: " + cash); // lexical scope access
```

```
  }
```

```
  son();
```

```
}
```

```
father();
```

Output Son spends father's cash: 500

2. Higher-Order Functions & Callbacks

Definitions

- **Callback Function:** Function passed as parameter argument into another function.
- **Higher-Order Function (HOF):** Function that takes callbacks or outputs new function structures.

Example

javascript

```
// Callback function
```

```
const displayOutput = (res) => console.log("Calculated output is: " + res);
```

```
// Higher-order function
```

```
function adder(num1, num2, callbackAction) {
```

```
    let sum = num1 + num2;
```

```
    callbackAction(sum); // Callback execution
```

```
}
```

```
adder(40, 60, displayOutput);
```

Output Calculated output is: 100

PART 7: PRACTICAL INTERVIEW PROBLEMS

Reverse a String

javascript

```
function reverseStr(val) {
```

```
    let result = "";
```

```
    for (let i = val.length - 1; i >= 0; i--) {
```

```
        result += val[i];
```

```
    }
```

```
    return result;
```

```
}
```



```
console.log(reverseStr("coder")); // redoc
```

Palindrome Checker

javascript

```
function isPalindrome(str) {  
    let clean = str.toLowerCase();  
    let rev = clean.split("").reverse().join("");  
    return clean === rev;  
}
```

```
console.log(isPalindrome("radar")); // true
```

Count Characters

javascript

```
function getCharCount(str) {  
    let count = 0;  
    for (let char of str) {  
        if (char !== " ") count++;  
    }  
    return count;  
}  
  
console.log(getCharCount("my code")); // 6
```

Anagram Basics

javascript

```
function anagramCheck(s1, s2) {  
    let sort1 = s1.split("").sort().join("");  
    let sort2 = s2.split("").sort().join("");  
    return sort1 === sort2;  
}  
  
console.log(anagramCheck("silent", "listen")); // true
```

Frequency Count

```
javascript

function frequencyMap(str) {

  let fMap = {};

  for (let item of str) {

    fMap[item] = (fMap[item] ?? 0) + 1;

  }

  return fMap;

}

console.log(frequencyMap("test")); // { t: 2, e: 1, s: 1 }
```



QUICK REVISION SUMMARY

- **Basic functions:** Code chunks encapsulated to perform reusable tasks.
 - **Expression & Anonymous:** Functions assigned inside variables, anonymous lacks explicit name identifier.
 - **Arrow function ES6:** Short syntax format, implicit return logic templates.
 - **Modern parameter features:** Default values handles omitted args; Rest ... packs inputs as array; Spread unpacks.
 - **Scopes:** Global (whole file), Function (local container), Block (only within curly brace blocks).
 - **Callbacks & HOF:** Callback is a function passed as argument; HOF is the receiving/returning function wrapper.
-



IMPORTANT DEFINITIONS (INTERVIEW-READY)

1. **DRY Principle:** "Don't Repeat Yourself" - A design approach recommending developers encapsulate repetitive logic into reusable functions.
2. **Callback Function:** A function passed as an input parameter argument to another function, executed when invoked inside the main function.
3. **Higher-Order Function:** A function that receives another function as an argument, or outputs a function as its result.

4. **Lexical Scope:** Scoping system where variable visibility is determined by the variable's physical declaration position in the source code nesting.
 5. **Function Hoisting:** JavaScript behavior where function declarations are moved to the top of the current scope during compile phase, allowing them to be run before their physical line declaration.
-

IMPORTANT INTERVIEW QUESTIONS & ANSWERS

Q1. Difference between Function Declaration and Function Expression?

Ans: Declarations are hoisted and can be called before their definition. Expressions are variable assignments, so calling them before declaration throws a `ReferenceError`.

Q2. What are arrow functions and when to avoid them?

Ans: Compact functions syntax using `=>`. Avoid them as method definitions inside objects because they do not bind their own `this` context.

Q3. What are Anonymous Functions?

Ans: Functions without a name identifier. Mostly used as inline arguments or callbacks.

Q4. Difference between Parameters and Arguments?

Ans: Parameters are variables in function definitions. Arguments are the actual values passed during invocation.

Q5. Predict output:

```
javascript
function test(a, b = 2) {
    return a * b;
}
console.log(test(5));
```

Ans: 10. Parameter `b` falls back to 2 because no second argument was passed.

Q6. How does Rest parameters work?

Ans: Rest parameter (`...nums`) groups multiple arguments into a single array parameter.

Q7. What will happen if you place code after return statement?

Ans: That code becomes unreachable and will not be executed.

Q8. Predict output:

javascript

```
const myFunc = () => {  
  let a = 10;  
  return a;  
};  
console.log(myFunc());
```

Ans: 10. Standard arrow function execution.

Q9. What is Block Scope?

Ans: Variables defined with let and const inside {} block are accessible only inside those curly braces.

Q10. What is global scope variables risk?

Ans: Global variables can be modified from anywhere in the codebase, leading to naming collisions and buggy state updates.

Q11. Predict output:

javascript

```
let name = "Ravi";  
  
function show() {  
  let name = "Rahul";  
  console.log(name);  
}  
  
show();
```

Ans: "Rahul". The function-scoped local variable shadows the global variable.

Q12. Predict output:

javascript

```
function outer() {  
  let count = 0;  
  function inner() {  
    count++;  
    console.log(count);  
  }  
}
```

```
    }  
    inner();  
  }  
  outer();
```

Ans: 1. Inner nested function accesses outer function's variable count via lexical scope.

Q13. Explain Higher-Order function using array helper function.

Ans: map() is a Higher-Order Function because it accepts a callback function to transform array elements.

Q14. Predict output:

```
javascript  
  
const sum = (a, b) => a + b;  
  
console.log(typeof sum);
```

Ans: "function". Arrow functions have the type "function".

Q15. Can Rest parameter be placed at beginning of parameter list?

Ans: No. Rest parameter must be the last parameter in the function declaration.

Q16. Predict output:

```
javascript  
  
let arr = [1, 2];  
  
console.log(...arr);
```

Ans: 1 2. Spread operator unpacks elements into separate values.

Q17. What is Lexical Scope?

Ans: Lexical scope determines variable accessibility based on the physical position of declarations in the source code.

Q18. Predict output:

```
javascript  
  
let x = 10;  
  
if (true) {  
    let x = 20;  
}
```

```
console.log(x);
```

Ans: 10. The local variable x = 20 inside the if block is scoped to that block only.

Q19. What are callbacks commonly used for?

Ans: Async event handlers, timers (setTimeout), and array operations.

Q20. Predict output:

```
javascript
```

```
let func = (a) => a;
```

```
console.log(func());
```

Ans: undefined. Since no argument was passed, parameter a is assigned undefined.



PRACTICE QUESTIONS

Question 1: Check Palindrome

- **Question:** Create a function to check if a word matches its reverse.
- **Solution:**

```
javascript
```

```
function isPal(str) {  
  let rev = str.split('').reverse().join('');  
  return str === rev;  
}
```

```
console.log(isPal("racecar")); // true
```

- **Explanation:** Compares clean string inputs with their reversed output.

Question 2: Find elements sum using Rest Parameters

- **Question:** Add any number of inputs passed to the function.
- **Solution:**

```
javascript
```

```
function sumUp(...nums) {  
  return nums.reduce((acc, current) => acc + current, 0);  
}
```

```
console.log(sumUp(1, 2, 3, 4)); // 10
```

- **Explanation:** Uses rest to group arguments, then sums them up.

Question 3: Character frequency mapper

- **Question:** Map occurrences of letters inside "apple".
- **Solution:**

javascript

```
function getFreq(word) {  
  let mapper = {};  
  for (let char of word) {  
    mapper[char] = (mapper[char] ?? 0) + 1;  
  }  
  return mapper;  
}  
  
console.log(getFreq("apple"));
```

- **Output:** { a: 1, p: 2, l: 1, e: 1 }
- **Explanation:** Checks matching keys in map and increments count. }

6:32 PM